Prompt AI for Web App Creation (version 2)

Build me an interactive web app for my high school [SUBJECT, e.g., Biology] students on [TOPIC/STANDARD]. My students are in a virtual, asynchronous program. Include [NUMBER] scored questions: a mix of multiple choice, short "explain your thinking" responses, and interactive activities.

Source materials (pull questions from these, and add your own varieties):
- Questions/slides/worksheet: [LINK]
- Simulation to recreate (if any): [LINK]

==========
1. WARM-UP AND LEVEL CHOICE
==========
- Start with 3 warm-up questions on the topic. These do NOT count toward the score. Show brief feedback after each one.
- Based on the warm-up, the app recommends one of three levels: Support, On-Level, or Challenge, with a short, encouraging explanation of why.
- The student makes the final choice and can pick any level.
- Record both the recommended level and the level the student chose.

==========
2. THREE LEVELS: SAME CONTENT, DIFFERENT SUPPORT
==========
- All three levels must cover the exact same content, learning targets, and number of questions. They must be equally rigorous and scored the same way. No level should be easier or shorter.
- The levels differ ONLY in how the content is expressed and how much help is offered:
  - Support: plainer language, broken into smaller steps, sentence starters shown right away, a word bank, visuals or worked examples, and 3 hints per question.
  - On-Level: standard wording, 2 hints, and a short checklist for written responses.
  - Challenge: more academic wording, 2 brief hints, and prompts to make connections or apply the idea to new situations.
- On every level: tiered hints (hint 1 = nudge, hint 2 = more help), vocabulary pop-ups for key terms, and read-aloud. Log how many hints each student used.
- Write TWO versions of every scored question (same skill, different details). Randomly choose which version each student sees first.
- If a student misses a question, offer one retry with the OTHER version. If they miss the retry, show the key idea and the correct answer, then move on.
- Scoring (same on every level): 1 point for a correct first try, 1/2 point for a correct retry, 0 otherwise.
- Randomize answer choice order for each student. Randomize question order within each section, but keep sections in a logical learning order.

==========
3. DESIGN AND ENGAGEMENT
==========

- Create a visual theme based on the topic (for example, a cell biology lesson becomes a "mission inside the cell," an ecology lesson becomes a field expedition). Use the theme in the title, colors, icons, section names, and feedback messages.
- Do NOT make it plain or beige. Give the page a themed background: a soft gradient with a faint pattern of topic-related shapes behind the content, and a themed header. Questions sit on solid cards so text always has strong contrast.
- Give each section/station its own accent color on its question cards.
- Add a themed illustration on the start and finish screens.
- Make it multi-dimensional: include at least 2-3 interactive formats beyond multiple choice, such as drag-and-drop labeling, sorting or matching, a simple simulation or adjustable model, data or graph interpretation, or a clickable diagram. Every drag-and-drop activity must also have a click or keyboard alternative.
- Show progress with a themed progress bar or map.
- Give positive, specific feedback after answers, tied to the theme.
- Gentle themed confetti when a student gets an answer right (warm-ups and scored questions) and at completion. It falls slowly and fades in about 3 seconds, can be turned off in Reading settings, and never shows for students whose device is set to reduce motion.
- The theme, visuals, and interactive formats must be the SAME across all three levels. Only the wording and support change.
- Keep the content front and center. Decorations should never crowd or distract from the questions.
- No flashing or fast animations. Keep motion subtle, and no sound effects.
- Keep it lightweight so it runs smoothly on Chromebooks.

==========
4. BUILT-IN SIMULATION (if the lesson uses one)
==========
- Do NOT link or embed the outside simulation. Students must never need another tab. Recreate the simulation's functionality inside the app (it can look different, and better). Include every control and tab the original has that the questions depend on.
- A "Sim Lab" button in the header opens the simulation on every screen.
- When the Sim Lab is open, show a banner at the top ("Your mission continues below: Question 5 of 12") and a large button at the bottom that closes it and takes the student to their next step (for example, "Close the Sim Lab and start the warm-up").
- Include an "Explore challenges" list inside the Sim Lab based on the source questions.
- Make what is happening visible: for example, dissolved particles spread through the liquid so students can see how many there are and how dense they are, and they get denser as liquid is removed.
- Any on/off or yes/no state (a light bulb, a switch, a reading) must be obvious at a glance: a clear color change PLUS a text label (like "Bulb ON" / "Bulb OFF") PLUS a level meter, never a subtle shade.
- Live text readouts of all values, a text description for screen readers, a "Pause motion" button, and Reset.
- Using the Sim Lab counts as activity, not inactivity.

5. ACCESSIBILITY AND DYSLEXIA-FRIENDLY DESIGN

Follow WCAG 2.1 AA accessibility guidelines. Build these features in for ALL students, so no student has to be identified to use them:

Text and layout
- Use a clear sans-serif font such as Lexend or Atkinson Hyperlegible. Base text at least 18px.
- Line spacing of about 1.5, slightly increased letter spacing, and extra space between paragraphs.
- Left-aligned text (never justified). Keep lines short (about 60-70 characters).
- Use bold for emphasis. Avoid italics, underlining (except links), and ALL CAPS.
- Cards use a soft off-white or cream background with dark gray text instead of pure black on bright white. Maintain strong contrast.
- Break text into short chunks, bullet points, and numbered steps. One question per screen.
- Use icons and visuals alongside key words. Never rely on color alone to show meaning (for example, use a check mark and a color).

Reading settings panel (a button visible on every screen)
- Text size: larger or smaller.
- Spacing: normal or extra.
- Background color: cream, light blue, light gray, or dark mode. Each choice gets its own matching themed background.
- Optional dyslexia font toggle (for example, OpenDyslexic).
- Plain background toggle (removes the pattern).
- Confetti on/off.
- Settings stay in place for the whole activity.
- Check that every button and label is easy to read in every background color, including dark mode.

Read-aloud
- A read-aloud button on every question, answer choice, hint, and passage, with word-by-word highlighting as it reads.
- Adjustable reading speed.

Responses
- Do not penalize spelling in short-answer responses. Score for science understanding by checking for the key ideas.
- Allow voice typing (Chromebook dictation) in answer boxes.
- Offer sentence starters as an option on every level (more prominent on Support).

Screen reader and keyboard
- Everything works with keyboard only (Tab, Enter, arrow keys), with a clear visible focus outline.
- All images, diagrams, and simulations have descriptive alt text or text descriptions.
- Use proper headings, labels, and ARIA labels so screen readers work.
- Captions for any video and transcripts for any audio.

No time pressure
- No countdown timers or speed-based scoring.

==========
6. ACADEMIC INTEGRITY
==========
- At the start, show a short notice: "This activity records time on task, time away from the page, and inactivity." The student types their first and last name to begin. Do not ask for a class period.
- Block paste, drag-and-drop text, and right-click in all answer boxes. Block selecting and copying of the question text. Show a friendly message if they try.
- Voice typing (dictation) must NOT be blocked or counted as pasting.
- Track every time the student leaves the page or tab: count it and time how long they were gone. Use page visibility (switching tabs, minimizing) as the main signal, so clicking on the Canvas menu around the app is not counted as leaving. When they return, show: "I noticed you left the page. This has been recorded."
- If the student is inactive for [3] minutes, pause and gently ask "Why were you inactive?" with choices (thinking, reading, needed help, tech issue, other) plus a short text box. Record their answer.
- Using read-aloud, the reading settings panel, hints, vocabulary pop-ups, or the Sim Lab counts as ACTIVITY, not inactivity.
- Count any attempt to paste, and log any answer where a large amount of text appears at once from pasting (not from typing or dictation).
- Warn the student before they close the page mid-activity.

==========
7. RESULTS CARD AND SCREENSHOT (shown at the end)
==========
- Students turn in a screenshot of their results in Canvas, so the end screen starts with a compact results card that fits on one screen inside a 900px-tall iframe. It shows:
  - The student's FULL NAME in large text and the date and time finished
  - Score (out of [NUMBER] and as a percent), and the score for each section/station
  - Level chosen (and the suggested level if different), hints used, total time, times away, time away, inactivity check-ins
  - A short completion code (like "FNW-J7M") made from the name, finish time, and score, so I can match a screenshot to my spreadsheet
  - The save status
- Below the card, a "Turn in your results" box with read-aloud explains: press Ctrl + Shift + Show windows and drag around the card; paste it into the Canvas assignment with Ctrl + V; if pasting doesn't work, upload it from the Downloads folder; then press Submit.
- Below that, the themed celebration message and illustration.

==========
8. HOSTING, SAVING RESULTS, AND CANVAS
==========
- I host each app in Google Apps Script. Give me ONLY one complete file named index.html (starting with <!DOCTYPE html>, all CSS and JavaScript

inside it). My Code.gs is already set up: it serves the page, allows Canvas embedding, and saves each activity to its own tab in my results Google Sheet.
- Put a constant at the top of the script: ACTIVITY_NAME = "[ACTIVITY NAME]".
- When the student finishes, send results with google.script.run.saveResult(data), where data uses exactly these fields: activityName, name, started, finished (ISO date strings), recommendedLevel, chosenLevel, score (a percent number), hintsUsed, timesAway, timeAwayMinutes, timeOnTaskMinutes, inactivityReasons (list), pasteAttempts, readingToolsUsed (list), shortAnswers (list, each starting with the question number, then the version, try number, whether it met the key ideas, any large-text flag, and the full answer), completionCode.
- Show "Your results were saved" on success. If saving fails, show a friendly message and a "Try again" button. Always show the results card either way.
- If google.script is not available (previewing outside Apps Script), skip saving without errors.
- The app runs inside sandboxed iframes (Apps Script + Canvas), which block form submissions. Do NOT use <form> elements or submit buttons. Use buttons with click handlers, and let Enter work in the name boxes.
- Do not send any student data anywhere except google.script.run. No AI or outside services. Load only fonts from Google Fonts or jsDelivr.
- The app will be embedded in a Canvas page in an iframe about 900px tall. Make the layout fit and scroll well inside it on a Chromebook.
- Before giving me the file, test the whole flow: start, Sim Lab, warm-up, level choice, every question type, retries, the results card, and saving (success and failure).

Keep it student-friendly and fun, but always clear and accessible.
Do not ask me for any student names.